

High-Performance Code Optimizations with Accuracy Guarantees

Neea Rusch¹

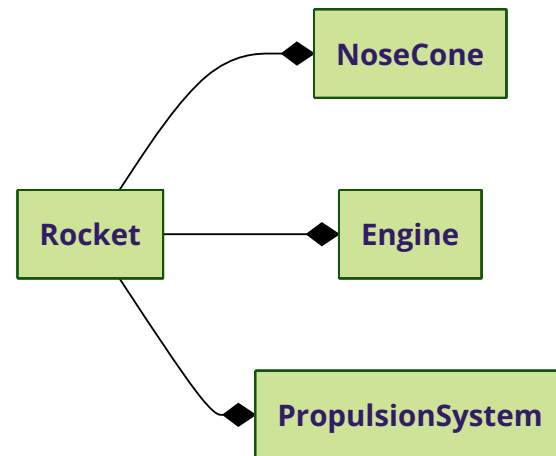
j.w.w. Eva Darulova¹ and Thomas Kœhler²

¹Uppsala University, Sweden

²CNRS; Université de Strasbourg, France

Computing is the study of reusable pure abstractions.

- Examples: integers, lists, protocols, programming languages, etc.
- Abstractions have algorithmic value in designing, representing, and reasoning about computational processes.

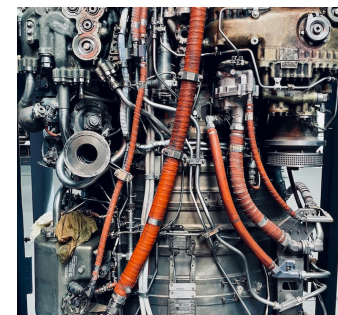
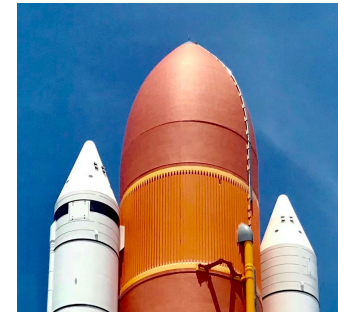


Most models in natural sciences and engineering are based on real numbers.

Despite progress, a significant gap remains between the well-established mathematical theory of real numbers and continuous structures and the discrete computation by modern computers.⁴

Computation of and on real numbers is needed, and is of theoretical and practical importance.

⁴Vasco Brattka on “Computability and Complexity in Analysis”, <https://cca-net.de>.



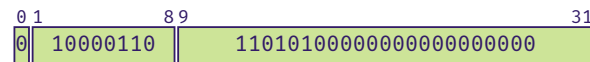
Real numbers are most commonly represented as floating-point (FP) arithmetic.⁵

- FPs are finite approximations with precision,
 - e.g., 32-bit single and 64-bit double precision.
- Handle large ranges, but introduce rounding errors.
 - Arithmetic operations accumulate errors.
- Some special values:
 - Unrepresentable result NaN, not a number.
 - Numeric values strictly bounded by $-\infty$ and $+\infty$.
 - Signed zero, ± 0 .

Common Representation

$$\pm(1 + \text{mantissa})2^{\text{exponent}}$$

Example: single-precision (float)



1-bit sign, 8-bit exponent,
23-bit mantissa

⁵Working Group for Floating-Point Arithmetic. (2019). "IEEE 754-2019" [Technical report]. <https://standards.ieee.org/ieee/754/6210>

For example, in single instruction, multiple data (SIMD) parallel processing:

"SIMD instructions, whose width is a predetermined byte size, can typically process twice as many floats as doubles per cycle. Therefore, performance-sensitive applications are very likely to favor lower-precision alternatives when implementing their algorithms⁶."

⁶Courbet, C. (2021). NSan: a floating-point numerical sanitizer. *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*, 83–93. <https://doi.org/10.1145/3446804.3446848>

Question

How can we actually understand and study the tradeoffs between floating-point accuracy and program efficiency?

- How to define and quantify the tradeoffs?
- What is an appropriate common reasoning model for the two concepts?
- Do the quality characteristics carry to executable code?
- Is the analysis sustainable over software lifecycle changes?



Our solution

Unify existing frameworks that in isolation handle part of the challenge well.

1. ***RISE** & Shine*: a compiler ecosystem for code performance optimization.
2. *Daisy*: an analysis framework for reasoning about accuracy and rounding error.

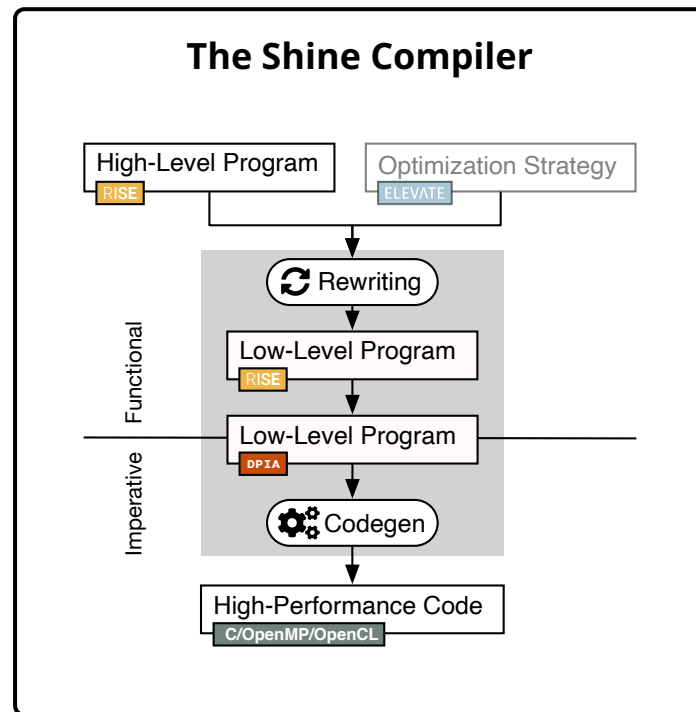
This will allow us to quantify effects of accuracy on performance, and vice versa.

Importantly, the combination is more powerful than its individual parts.

RISE & Shine

A source-to-source compiler for high-performance code.

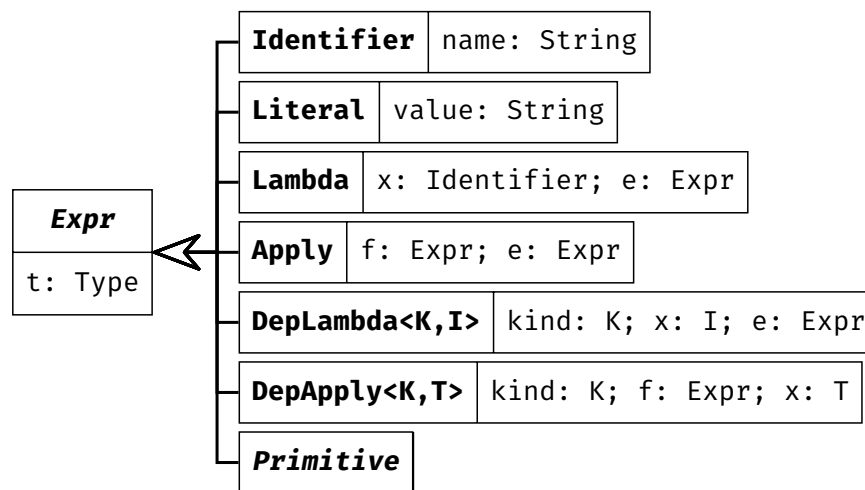
1. The inputs are a high-level program and a separate optimization strategy.
2. Shine rewrites the high-level program, based on the optimization strategy, into a low level program.
3. The low-level program encodes all implementation and optimization decisions explicitly.
4. Codegen translates the low-level program into high-performance parallel C or OpenCL code.



RISE is a functional pattern-based data-parallel language.

- A functional programming language, based on well-known data-parallel patterns, like map/reduce, without recursion.
- Programs convey functional behavior, with the scheduling strategy defined separately.
- The language is powerful enough to express many practical scientific computational kernels.

RISE grammar



primitives include:

map, filter, zip, reduce,...

A high-level program for computing the arithmetic mean.

```
def average = depFun((N: Nat) =>
  fun((N `.` f64) ->: f64)(arr =>
    reduce(add)(lf64(0))(arr)
    / (length(arr) |> cast)
  )
)
```

- Input is a double-type vector of size N.
- The reduce handles summation, and the last line (length... cast) divides.

Programs at different compilation stages: *optimizing matrix computations*

High-level RISE

```
def highLevelProgram: ToBeTyped[Rise] =  
  depFun((n: Nat, m: Nat, o: Nat) =>  
    fun(n`.o`.f32)(A => fun(m`.o`.f32)  
      (B =>  
        A |> map(fun(rowOfA =>  
          B |> map(fun(rowOfB =>  
            zip(rowOfA)(rowOfB) |>  
              map(fun(x => fst(x) * snd(x)))  
                |> reduce(add)(l(0.0f)) ))))))))
```

(optimization strategy implied)



Low-level RISE

```
def lowLevelProgram: ToBeTyped[Rise] =  
  depFun((n: Nat, m: Nat, o: Nat) =>  
    fun(n`.o`.f32)(A => fun(m`.o`.f32)  
      (B =>  
        A |> mapPar(fun(rowOfA =>  
          B |> mapSeq(fun(rowOfB =>  
            zip(rowOfA)(rowOfB) |>  
              map(fun(x => fst(x) * snd(x)))  
                |> reduceSeq(add)(l(0.0f)) ))))))))
```



Generated High Performance OpenMP Code

```
#include <stdint.h>  
  
void foo(  
  float* output, int n, int m,  
  int o, float* A, float* B)  
{  
  #pragma omp parallel for  
  for (int i = 0; i < n; i = 1 + i) {  
    for (int j = 0; j < m; j = 1 + j) {  
      float acc = 0.0f;  
      for (int k = 0; k < o; k = 1 + k) {  
        acc = acc + A[k + i * o] * B[k + j * o];  
      }  
      output[j + i * m] = acc;  
    }  
  }  
}
```

The RISE & Shine compiler¹⁰

- *Separate concerns* – Computations are expressed as high-level abstractions, and should not be changed to express optimizations.
- *Facilitate reuse* – Defining optimization strategies separately from the computational program facilitates reuse of programs and strategies.
- *Enable compositionality* – Computations and strategies are written as compositions of user-defined “blocks.”
- *Support formal reasoning* – Computational patterns and strategies have well-defined semantics for formal reasoning.

¹⁰Hagedorn, B., Lenfers, J., Köhler, T., Qin, X., Gorlatch, S., & Steuwer, M. (2023). Achieving High Performance the Functional Way: Expressing High-Performance Optimizations as Rewrite Strategies. *Communications of the ACM*, 66(3), 89–97. <https://doi.org/10.1145/3580371>

RISE & Shine employs functional programming techniques to solve the high performance code generation and scheduling challenge with elegance.

but accuracy of generated code is out of scope

Judging accuracy poses many challenges.

- Static analysis must deduce sound but informative error bounds.
- The ideal, infinite-precision error bounds are often not available for comparison.
- Analysis considers straight-line programs, which has technical implications and affects analysis scalability in practice.
- Handling non-straight-line programs is an open challenge.

In response, several theories and implementations exist. They have complementary capabilities, but the overall ‘tooling landscape’ is fragmented.



Daisy

An accuracy-focused modular framework for finite-precision computations.

Daisy provides a unified and extensible environment, for verification and development of program analyses and optimizations, specialized in accuracy.

Some built-in features:

- Multiple number system support
- Sound roundoff error analysis
- Rewriting optimizations
- Transcendental function support
- SMT solver interfaces
- Code generation, etc.

```
import daisy.lang._
import Real._

object Object {
  def function1(x: Real): Real = {
    require(0 <= x && x <= 1)

    val z = x + x
    z * x

  } ensuring(res => res +/- 1e-5)
}
```

The input language is a subset of Scala.

Ranges of all input variables must be defined: `require` includes the specification of input values.

The ensuring clause contains an assertion of the absolute roundoff error of magnitude e on variable x .

For numerical programs over array-like data structures, it is possible to leverage functional programming concepts to improve quality and efficiency of accuracy analysis.

- The functional primitives carry semantic information
- E.g. the `map` function does not propagate errors between iterations
 - Analyzing the loop body once reduces rounding error
 - Unrolling fewer operations avoids redundant analysis steps and improves scalability
- The idea is generally applicable to programs expressed in a functional style

This technique has been shown to be effective in Daisy.

Daisy provides an adjustable and extendable setting for accuracy analysis, and in this setting, functional programming techniques offer tangible benefits.

but efficiency of the analyzed code is out of scope

Rise & Shine + Daisy



+



⇒

?

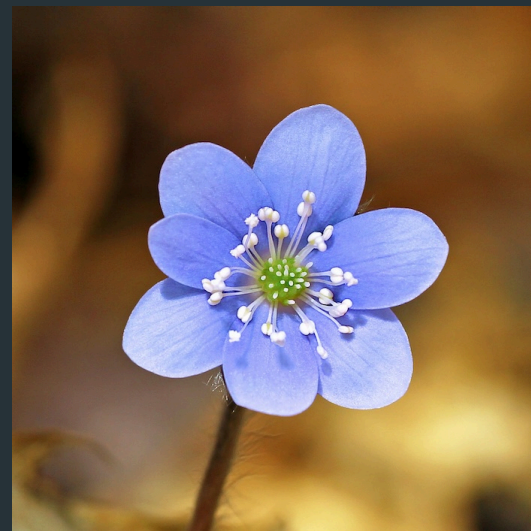
Rise & Shine + Daisy



+



⇒



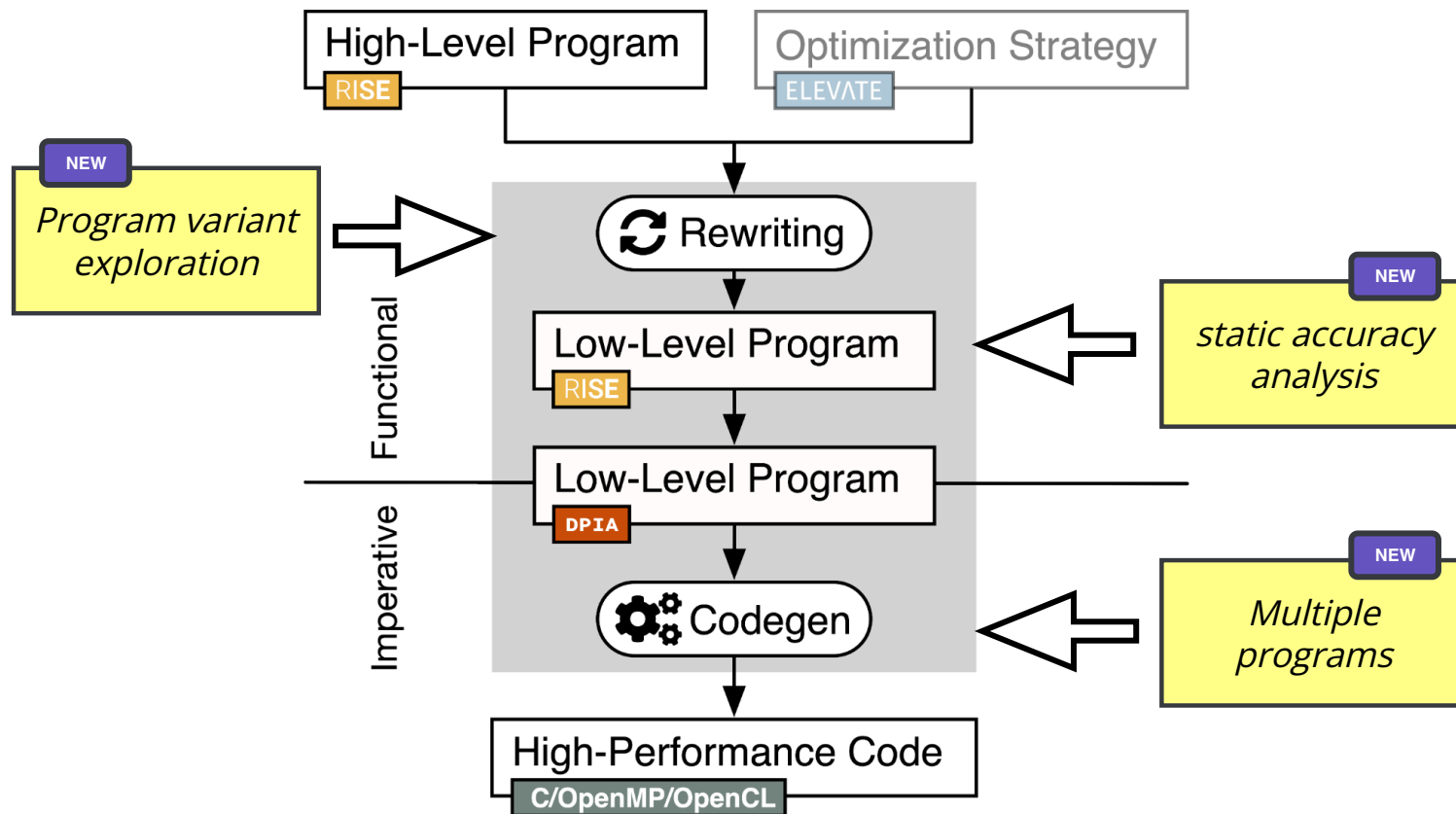
Hepatica



The Hepatica project merges **RISE** & Shine and Daisy to enable concurrent reasoning about tradeoffs between accuracy and performance.

The merger is not immediate – some requirements:

- Must have a common model of reasoning, i.e., shared programming language.
- Must implement accuracy analysis on **RISE** and add program specifications.
- Must develop effective strategies to explore program variants and choose ideal ones.
- Need abilities to quantify and measure impacts of high-level code changes.



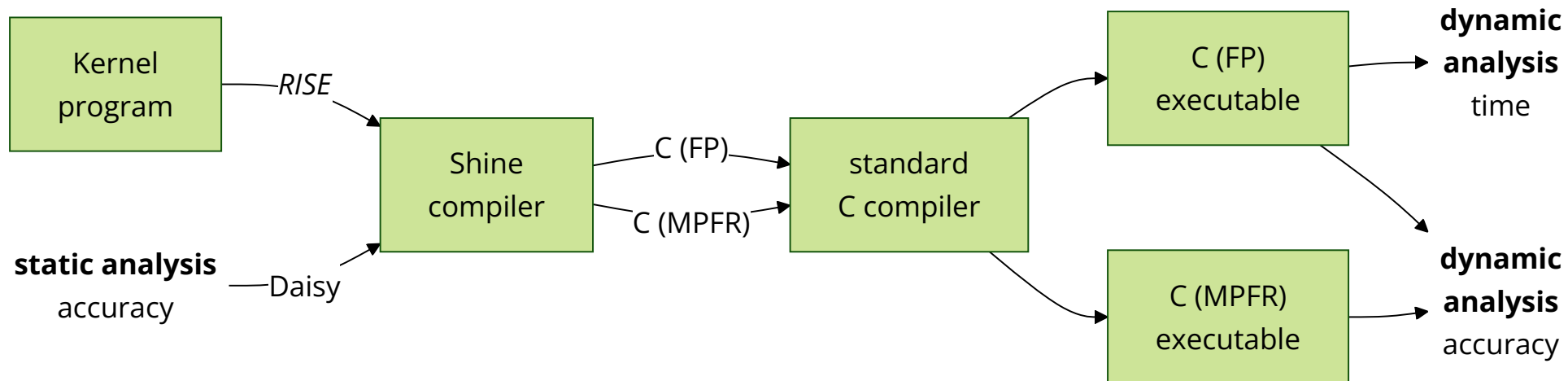
Our empirical setup considers two evaluation phases:

- **Static analysis** provides compilation-time feedback about accuracy.
 - Must be fast to assess numerous rewriting candidates.
- **Dynamic analysis** of executable code.
 - Codegen now produces a regular C (FP) program and a precise MPFR program.
 - The C program is measured for running time and rounding error bounds.
 - The MPFR execution provides the precise ideal for comparison.

Benchmarks (20+) will come from prior experiments on DS2L¹⁵ and RISE & Shine¹⁶.

¹⁵Isychev, A., & Darulova, E. (2023). Scaling up Roundoff Analysis of Functional Data Structure Programs. In M. V. Hermenegildo & J. F. Morales (Eds.), *Static Analysis: Static Analysis*. https://doi.org/10.1007/978-3-031-44245-2_17

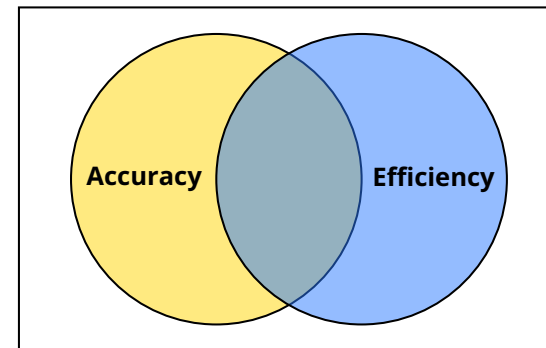
¹⁶Steuwer, M., Remmelg, T., & Dubach, C. (2017). LIFT: A functional data-parallel IR for high-performance GPU code generation. In (Ed.), *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO): 2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. <https://doi.org/10.1109/CGO.2017.7863730>



a.k.a. discussion topics

Identifying beneficial programming patterns

Are some programming patterns accurate and efficient, accurate or efficient, or neither?



Strategies for finding interesting program variants

- Tiling-based optimization improves cache locality; a potential performance boost.
- Reducing arithmetic operations is beneficial for accuracy.